

Mahi Prashant Nakhate

10

```
In [ ]: import pandas as pd
import numpy as np
import seaborn as sns

import matplotlib.pyplot as plt
%matplotlib inline

import warnings
warnings.filterwarnings("ignore")
```

```
In [10]: df = pd.read_csv('voice.csv')
df.head()
```

Out[10]:

	meanfreq	sd	median	Q25	Q75	IQR	skew	kurt	sp.ent	sfm	...	centroid	meanfun	minfun	maxfun
0	0.059781	0.064241	0.032027	0.015071	0.090193	0.075122	12.863462	274.402906	0.893369	0.491918	...	0.059781	0.084279	0.015702	0.275
1	0.066009	0.067310	0.040229	0.019414	0.092666	0.073252	22.423285	634.613855	0.892193	0.513724	...	0.066009	0.107937	0.015826	0.250
2	0.077316	0.083829	0.036718	0.008701	0.131908	0.123207	30.757155	1024.927705	0.846389	0.478905	...	0.077316	0.098706	0.015656	0.271
3	0.151228	0.072111	0.158011	0.096582	0.207955	0.111374	1.232831	4.177296	0.963322	0.727232	...	0.151228	0.088965	0.017798	0.250
4	0.135120	0.079146	0.124656	0.078720	0.206045	0.127325	1.101174	4.333713	0.971955	0.783568	...	0.135120	0.106398	0.016931	0.266

5 rows × 21 columns

```
In [11]: df.shape
```

Out[11]: (3168, 21)

```
In [12]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3168 entries, 0 to 3167
Data columns (total 21 columns):
#   Column      Non-Null Count  Dtype
---  -
0   meanfreq    3168 non-null   float64
1   sd          3168 non-null   float64
2   median      3168 non-null   float64
3   Q25         3168 non-null   float64
4   Q75         3168 non-null   float64
5   IQR         3168 non-null   float64
6   skew        3168 non-null   float64
7   kurt        3168 non-null   float64
8   sp.ent      3168 non-null   float64
9   sfm         3168 non-null   float64
10  mode        3168 non-null   float64
11  centroid    3168 non-null   float64
12  meanfun     3168 non-null   float64
13  minfun      3168 non-null   float64
14  maxfun      3168 non-null   float64
15  meandom     3168 non-null   float64
16  mindom      3168 non-null   float64
17  maxdom      3168 non-null   float64
18  dfrange     3168 non-null   float64
19  modindx     3168 non-null   float64
20  label       3168 non-null   object
dtypes: float64(20), object(1)
memory usage: 519.9+ KB
```

In [13]:

df.describe()

Out[13]:

	meanfreq	sd	median	Q25	Q75	IQR	skew	kurt	sp.ent	sfm	
count	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.000000	3168.00
mean	0.180907	0.057126	0.185621	0.140456	0.224765	0.084309	3.140168	36.568461	0.895127	0.408216	0.16
std	0.029918	0.016652	0.036360	0.048680	0.023639	0.042783	4.240529	134.928661	0.044980	0.177521	0.07
min	0.039363	0.018363	0.010975	0.000229	0.042946	0.014558	0.141735	2.068455	0.738651	0.036876	0.00
25%	0.163662	0.041954	0.169593	0.111087	0.208747	0.042560	1.649569	5.669547	0.861811	0.258041	0.11
50%	0.184838	0.059155	0.190032	0.140286	0.225684	0.094280	2.197101	8.318463	0.901767	0.396335	0.18
75%	0.199146	0.067020	0.210618	0.175939	0.243660	0.114175	2.931694	13.648905	0.928713	0.533676	0.22
max	0.251124	0.115273	0.261224	0.247347	0.273469	0.252225	34.725453	1309.612887	0.981997	0.842936	0.28

In [14]:

df.corr()

Out[14]:

	meanfreq	sd	median	Q25	Q75	IQR	skew	kurt	sp.ent	sfm	mode	centroid	meanfun
meanfreq	1.000000	-0.739039	0.925445	0.911416	0.740997	-0.627605	-0.322327	-0.316036	-0.601203	-0.784332	0.687715	1.000000	0.460844
sd	-0.739039	1.000000	-0.562603	-0.846931	-0.161076	0.874660	0.314597	0.346241	0.716620	0.838086	-0.529150	-0.739039	-0.466281
median	0.925445	-0.562603	1.000000	0.774922	0.731849	-0.477352	-0.257407	-0.243382	-0.502005	-0.661690	0.677433	0.925445	0.414909
Q25	0.911416	-0.846931	0.774922	1.000000	0.477140	-0.874189	-0.319475	-0.350182	-0.648126	-0.766875	0.591277	0.911416	0.545035
Q75	0.740997	-0.161076	0.731849	0.477140	1.000000	0.009636	-0.206339	-0.148881	-0.174905	-0.378198	0.486857	0.740997	0.155091
IQR	-0.627605	0.874660	-0.477352	-0.874189	0.009636	1.000000	0.249497	0.316185	0.640813	0.663601	-0.403764	-0.627605	-0.534462
skew	-0.322327	0.314597	-0.257407	-0.319475	-0.206339	0.249497	1.000000	0.977020	-0.195459	0.079694	-0.434859	-0.322327	-0.167668
kurt	-0.316036	0.346241	-0.243382	-0.350182	-0.148881	0.316185	0.977020	1.000000	-0.127644	0.109884	-0.406722	-0.316036	-0.194560
sp.ent	-0.601203	0.716620	-0.502005	-0.648126	-0.174905	0.640813	-0.195459	-0.127644	1.000000	0.866411	-0.325298	-0.601203	-0.513194
sfm	-0.784332	0.838086	-0.661690	-0.766875	-0.378198	0.663601	0.079694	0.109884	0.866411	1.000000	-0.485913	-0.784332	-0.421066
mode	0.687715	-0.529150	0.677433	0.591277	0.486857	-0.403764	-0.434859	-0.406722	-0.325298	-0.485913	1.000000	0.687715	0.324771
centroid	1.000000	-0.739039	0.925445	0.911416	0.740997	-0.627605	-0.322327	-0.316036	-0.601203	-0.784332	0.687715	1.000000	0.460844
meanfun	0.460844	-0.466281	0.414909	0.545035	0.155091	-0.534462	-0.167668	-0.194560	-0.513194	-0.421066	0.324771	0.460844	1.000000
minfun	0.383937	-0.345609	0.337602	0.320994	0.258002	-0.222680	-0.216954	-0.203201	-0.305826	-0.362100	0.385467	0.383937	0.339360
maxfun	0.274004	-0.129662	0.251328	0.199841	0.285584	-0.069588	-0.080861	-0.045667	-0.120738	-0.192369	0.172329	0.274004	0.311950
meandom	0.536666	-0.482726	0.455943	0.467403	0.359181	-0.333362	-0.336848	-0.303234	-0.293562	-0.428442	0.491479	0.536666	0.270844
mindom	0.229261	-0.357667	0.191169	0.302255	-0.023750	-0.357037	-0.061608	-0.103313	-0.294869	-0.289593	0.198150	0.229261	0.162160
maxdom	0.519528	-0.482278	0.438919	0.459683	0.335114	-0.337877	-0.305651	-0.274500	-0.324253	-0.436649	0.477187	0.519528	0.277900
dfrange	0.515570	-0.475999	0.435621	0.454394	0.335648	-0.331563	-0.304640	-0.272729	-0.319054	-0.431580	0.473775	0.515570	0.275150
modindx	-0.216979	0.122660	-0.213298	-0.141377	-0.216475	0.041252	-0.169325	-0.205539	0.198074	0.211477	-0.182344	-0.216979	-0.054850

In [15]:

df.isnull().sum()

Out[15]:

meanfreq	0
sd	0
median	0
Q25	0
Q75	0
IQR	0
skew	0
kurt	0
sp.ent	0
sfm	0
mode	0
centroid	0
meanfun	0
minfun	0
maxfun	0
meandom	0
mindom	0
maxdom	0
dfrange	0
modindx	0
label	0
dtype:	int64

In [17]:

```
print("Total number of labela: {}".format(df.shape[0]))
print("Number of male: {}".format(df[df.label == 'male'].shape[0]))
print("Number of female: {}".format(df[df.label == 'female'].shape[0]))
```

Total number of labela: 3168
Number of male: 1584
Number of female: 1584

```
In [18]: X = df.iloc[:, :-1]
X.head()
```

Out[18]:

	meanfreq	sd	median	Q25	Q75	IQR	skew	kurt	sp.ent	sfm	mode	centroid	meanfun	minfun
0	0.059781	0.064241	0.032027	0.015071	0.090193	0.075122	12.863462	274.402906	0.893369	0.491918	0.000000	0.059781	0.084279	0.015702
1	0.066009	0.067310	0.040229	0.019414	0.092666	0.073252	22.423285	634.613855	0.892193	0.513724	0.000000	0.066009	0.107937	0.015826
2	0.077316	0.083829	0.036718	0.008701	0.131908	0.123207	30.757155	1024.927705	0.846389	0.478905	0.000000	0.077316	0.098706	0.015656
3	0.151228	0.072111	0.158011	0.096582	0.207955	0.111374	1.232831	4.177296	0.963322	0.727232	0.083878	0.151228	0.088965	0.017798
4	0.135120	0.079146	0.124656	0.078720	0.206045	0.127325	1.101174	4.333713	0.971955	0.783568	0.104261	0.135120	0.106398	0.016931

```
In [22]: from sklearn.preprocessing import LabelEncoder
y = df.iloc[:, -1]

# Encode Label category
# male -> 1
# female -> 0

gender_encoder = LabelEncoder()
y = gender_encoder.fit_transform(y)
y
```

Out[22]: array([1, 1, 1, ..., 0, 0, 0])

```
In [23]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X)
X = scaler.transform(X)
```

```
In [30]: from sklearn.model_selection import train_test_split
X_train,X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)
```

```
In [32]: from sklearn.svm import SVC
from sklearn import metrics
svc = SVC() #Default hyperparameters
svc.fit(X_train, y_train)
y_pred = svc.predict(X_test)
print("Accuracy Score:")
print(metrics.accuracy_score(y_test, y_pred))
```

Accuracy Score:
0.9763406940063092

Different Hyperplane in SVC

```
In [39]: svc = SVC(kernel = 'linear')
svc.fit(X_train,y_train)
y_pred = svc.predict(X_test)
print('Accuarcy Score:')
print(metrics.accuracy_score(y_test, y_pred))
```

Accuarcy Score:
0.9779179810725552

```
In [40]: svc = SVC(kernel = 'rbf')
svc.fit(X_train,y_train)
y_pred = svc.predict(X_test)
print('Accuarcy Score:')
print(metrics.accuracy_score(y_test, y_pred))
```

Accuarcy Score:
0.9763406940063092

```
In [41]: svc = SVC(kernel = 'poly')
svc.fit(X_train,y_train)
y_pred = svc.predict(X_test)
print('Accuarcy Score:')
print(metrics.accuracy_score(y_test, y_pred))
```

Accuarcy Score:
0.9589905362776026

```
In [42]: svc = SVC(kernel = 'sigmoid')
svc.fit(X_train,y_train)
y_pred = svc.predict(X_test)
print('Accuarcy Score:')
print(metrics.accuracy_score(y_test, y_pred))
```

Accuarcy Score:
0.7933753943217665

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: